МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВАСТОПОЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
ВТШ «СПИ»

ОТЧЕТ

о выполнении лабораторной работы № 1

Тема: «МЕТОДЫ ОДНОМЕРНОЙ МИНИМИЗАЦИИ УНИМОДАЛЬНОЙ ФУНКЦИИ»

по дисциплине

«ПРИКЛАДНАЯ МАТЕМАТИКА В ИИ»

Вариант № 4

Выполнил:
Студент группы ИИ/б-25-6-о
Заварзин А. В.
Проверила:
к.т.н., доцент, доцент кафедры ПМиИ,
доцент кафедры ИТиС
Ченгарь О. В.

Севастополь, 2026

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. В MS Excel построить таблицу значений заданной функции и её график на произвольном отрезке.
2. На основании построенного графика определить отрезок [a; b] с предполагаемым экстремумом функции (минимум функции f(x)).
3. Определить локальный минимум функции на основе методов одномерной оптимизации, выполнив 3-4 итерации (с помощью MS Excel):

а) методом деления отрезка пополам;

б) методом золотого сечения (для нечётных вариантов);

в) методом Фибоначчи (для чётных вариантов);

4. Решить задачу всеми методами, используя Python.
5. Сравнить результаты расчётов, сделанные в MS Excel и Python

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ ПО ВАРИАНТУ

Для варианта № 4 дана следующая функция:

$$f(x) = x^4 + 8x^3 - 6x^2 - 72x + 90$$

Для вычислений также возьмём значение погрешности:

$$\varepsilon = 0{,}05$$

ХОД РАБОТЫ

Задание 0

Из данной для варианта №4 функции найдём точки экстремума.

Для начала найдём производную функции:

$$f'(x) = 4x^3 + 24x^2 - 12x - 72$$

Далее приравниваем производную к нулю:

$$4x^3 + 24x^2 - 12x - 72 = 0$$

Находим корни:

$$\begin{cases} x_1 = -6 \\ x_2 = -\sqrt{3} \\ x_3 = \sqrt{3} \end{cases}$$

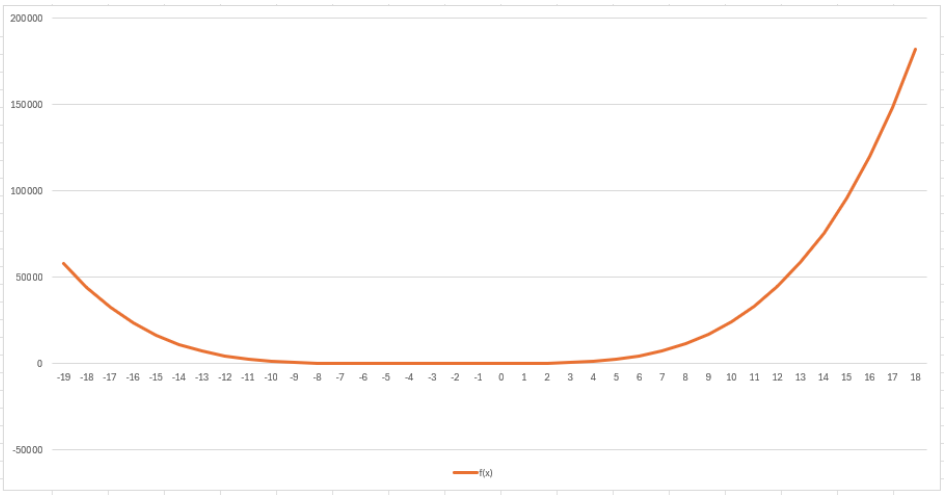
Подставляя все полученные значения в исходную функцию можно сказать, что точка минимума функции по оси располагается в -6. А значение функции в точке минимума равно -126.

Задание 1.

В начале необходимо построить таблицу значений функции в MS Excel, а затем, по полученной таблице, построить её график.

Результат построения таблицы значения и графика функции в среде MS Excel представлен на рисунке 2.1.

x	f(x)
-19	74741
-18	57762
-17	43797
-16	32474
-15	23445
-14	16386
-13	10997
-12	7002
-11	4149
-10	2210
-9	981
-8	282
-7	-43
-6	-126
-5	-75
-4	26
-3	117
-2	162
-1	149
0	90
1	21
2	2
3	117
4	474
5	1205
6	2466
7	4437
8	7322
9	11349
10	16770
11	23861
12	32922
13	44277
14	58274
15	75285
16	95706
17	119957
18	148482
19	181749



ε = 0,05

Рисунок 2.1 – Построенная таблица значений и график для функции

Задание 2

Исходя из полученного графика в задании 1 можно сказать, что минимум функции лежит в интервале [-10, 6].

Задание 3

После определения интервала в котором лежит предполагаемый минимум необходимо пройти 3 итерации с помощью методов деления отрезка пополам и Фибоначчи. Сначала был произведён поиск минимума функции с помощью метода деления отрезка пополам. Результаты первых 3 итераций представлены на рисунке 2.2.

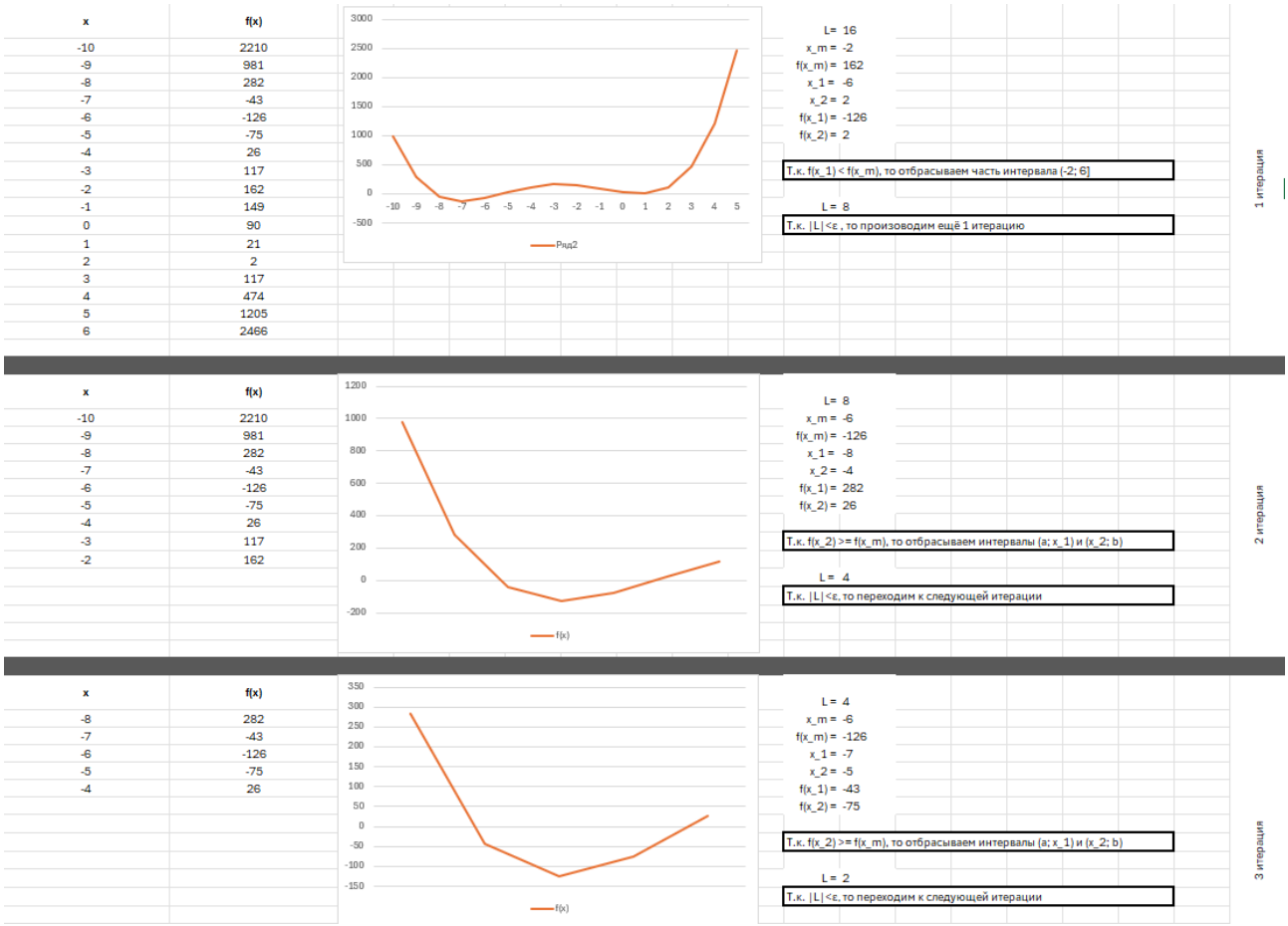


Рисунок 2.2 – Результаты первых 3 итераций метода деления отрезка пополам

Для расчётов были использованы инструменты MS Excel, а именно: обращение к элементам через ссылки, использование функций MS Excel для расчёта значения функции (функция СТЕПЕНЬ).

После произведения расчётов по методу деления отрезка пополам необходимо произвести 3 итерации по методу Фибоначчи.

Перед произведением вычислений по методу Фибоначчи необходимо построить таблицу чисел Фибоначчи. Часть таблицы чисел Фибоначчи, построенная средствами MS Excel представлена на рисунке 2.3.

1	F_0
1	F_1
2	F_2
3	F_3
5	F_4
8	F_5
13	F_6
21	F_7
34	F_8
55	F_9
89	F_10
144	F_11
233	F_12
377	F_13
610	F_14
987	F_15
1597	F_16
2584	F_17
4181	F_18
6765	F_19
10946	F_20
17711	F_21
28657	F_22
46368	F_23
75025	F_24
121393	F_25

Рисунок 2.3 – Часть таблицы чисел Фибоначчи

После построения таблицы чисел Фибоначчи можно произвести поиск минимума функции по методу Фибоначчи. Первые 4 итерации метода Фибоначчи представлены на рисунке 2.4.

Итерация	n	a	b	x_1	x_2	f(x_1)	f(x_2)
1	14	-10	6	-3,88854	-0,11146	37,50627	97,93959
2	13	-10	-0,11146	-6,22291	-3,88854	-122,541	37,50627
3	12	-10	-3,88854	-7,66563	-6,22291	138,7382	-122,541
4	11	-7,66563	-3,88854	-6,22291	-5,33126	-122,541	-101,069

Рисунок 2.4 – Первые 4 итерации метода Фибоначчи

Задание 4

После изучения алгоритмов всех методов и произведения вычислений в MS Excel необходимо написать код на языке Python, который будет находить точку минимума функции с точностью $\varepsilon = 0{,}05$ по всем трём методам, а также для удобства, можно реализовать построение графиков с показанной на них функцией и отмеченной точкой минимума. Для построения графиков будет использована библиотека «matplotlib», а для числовых вычислений (например, генерация равномерно распределённого интервала) будет использована библиотека «NumPy». Листинг кода представлен в приложении А.

Результаты работы программы представлены на рисунках 2.5 – 2.10.

```
======== МЕНЮ ========
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 1
 Итерация        a       b     x_1   f(x_1)     x_2   f(x_2)     x_m   f(x_m)
        1 -10.0000 -2.0000 -6.0000 -126.000  2.0000    2.000 -6.0000 -126.000
        2  -8.0000 -4.0000 -8.0000  282.000 -4.0000   26.000 -6.0000 -126.000
        3  -7.0000 -5.0000 -7.0000  -43.000 -5.0000  -75.000 -6.0000 -126.000
        4  -6.5000 -5.5000 -6.5000 -107.438 -5.5000 -111.438 -6.0000 -126.000
        5  -6.2500 -5.7500 -6.2500 -121.621 -5.7500 -122.121 -6.0000 -126.000
        6  -6.1250 -5.8750 -6.1250 -124.937 -5.8750 -125.000 -6.0000 -126.000
        7  -6.0625 -5.9375 -6.0625 -125.738 -5.9375 -125.746 -6.0000 -126.000
        8  -6.0312 -5.9688 -6.0312 -125.935 -5.9688 -125.936 -6.0000 -126.000
        9  -6.0156 -5.9844 -6.0156 -125.984 -5.9844 -125.984 -6.0000 -126.000


Результат вычисления по методу деления интервала:
    Точка минимума: x_min = -6.000000
    Значение в точке минимума: f(x_min) = -126.000000
    Интервал (границы): [-6.015625; -5.984375]
    Длина интервала: 0.031250
    Заняло итераций: 9
============================================================
```

Рисунок 2.5 – Результат вычисления по методу деления интервалов с помощью программы на языке Python

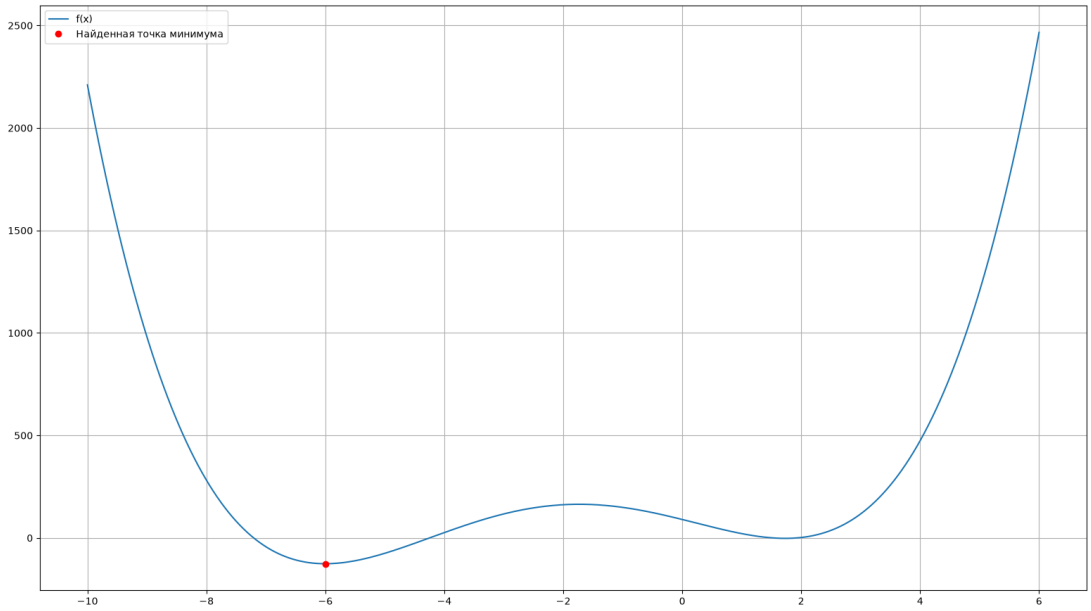


Рисунок 2.6 – График функции с найденной точкой минимума для метода деления интервалов

```
======== МЕНЮ ========
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 2
 Итерация        a       b     x_1   f(x_1)     x_2   f(x_2)
        1 -10.0000  6.0000 -3.8885   37.506 -0.1115   97.939
        2 -10.0000 -0.1115 -6.2229 -122.541 -3.8885   37.506
        3 -10.0000 -3.8885 -7.6656  138.740 -6.2229 -122.541
        4  -7.6656 -3.8885 -6.2229 -122.541 -5.3313 -101.070
        5  -7.6656 -5.3313 -6.7740  -78.684 -6.2229 -122.541
        6  -6.7740 -5.3313 -6.2229 -122.541 -5.8824 -125.112
        7  -6.2229 -5.3313 -5.8824 -125.112 -5.6718 -119.446
        8  -6.2229 -5.6718 -6.0124 -125.990 -5.8824 -125.112
        9  -6.2229 -5.8824 -6.0929 -125.418 -6.0124 -125.990
       10  -6.0929 -5.8824 -6.0124 -125.990 -5.9628 -125.910
       11  -6.0929 -5.9628 -6.0433 -125.875 -6.0124 -125.990
       12  -6.0433 -5.9628 -6.0124 -125.990 -5.9938 -125.997
       13  -6.0124 -5.9628 -5.9938 -125.997 -5.9814 -125.977
       14  -6.0124 -5.9814 -6.0000 -126.000 -5.9938 -125.997


Результат вычисления по методу Фибоначчи:
    Точка минимума: x_min = -6.000000
    Значение в точке минимума: f(x_min) = -126.000000
    Интервал (границы): [-6.012384; -5.981424]
    Длина интервала: 0.030960
    Заняло итераций: 14
============================================================
```

Рисунок 2.7 – Результат вычисления по методу Фибоначчи с помощью программы на языке Python

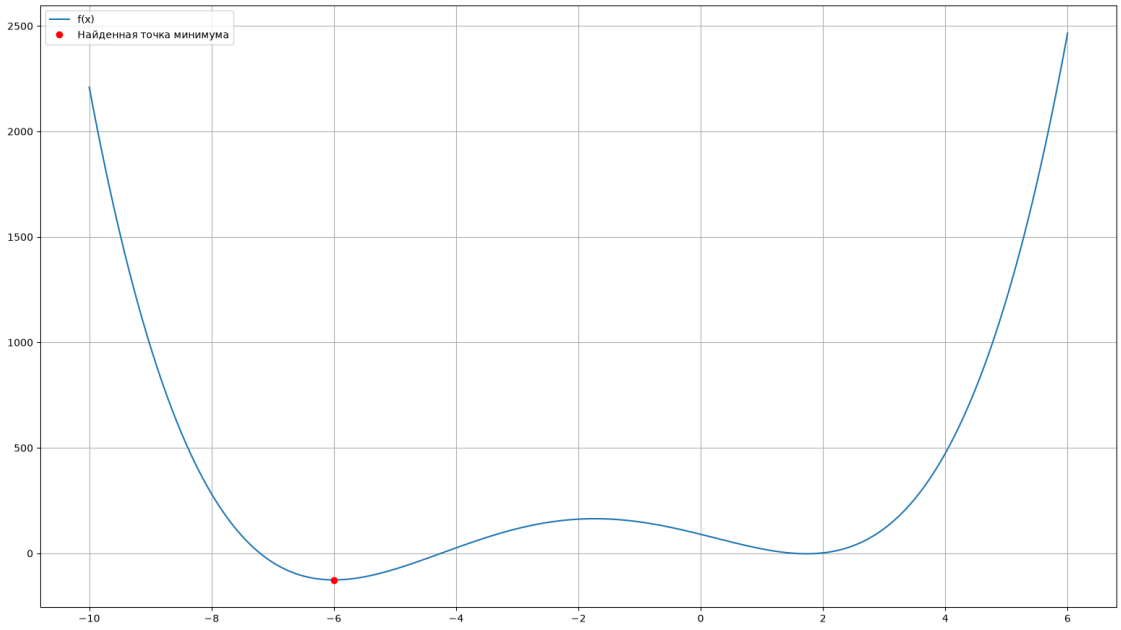


Рисунок 2.8 – График функции с найденной точкой минимума для метода Фибоначчи

```
======== МЕНЮ ========
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 3
 Итерация        a       b     x_1   f(x_1)     x_2   f(x_2)
        1 -10.0000 -0.1115 -6.2229 -122.541 -3.8885   37.513
        2 -10.0000 -3.8885 -7.6656  138.721 -6.2229 -122.541
        3  -7.6656 -3.8885 -6.2229 -122.541 -5.3312 -101.066
        4  -7.6656 -5.3312 -6.7739  -78.693 -6.2229 -122.541
        5  -6.7739 -5.3312 -6.2229 -122.541 -5.8823 -125.111
        6  -6.2229 -5.3312 -5.8823 -125.111 -5.6718 -119.445
        7  -6.2229 -5.6718 -6.0124 -125.990 -5.8823 -125.111
        8  -6.2229 -5.8823 -6.0928 -125.419 -6.0124 -125.990
        9  -6.0928 -5.8823 -6.0124 -125.990 -5.9627 -125.909
       10  -6.0928 -5.9627 -6.0431 -125.876 -6.0124 -125.990
       11  -6.0431 -5.9627 -6.0124 -125.990 -5.9934 -125.997
       12  -6.0124 -5.9627 -5.9934 -125.997 -5.9817 -125.978


Результат вычисления по методу золотого сечения:
    Точка минимума: x_min = -5.987549
    Значение в точке минимума: f(x_min) = -125.989798
    Интервал (границы): [-6.012400; -5.962697]
    Длина интервала: 0.049703
    Заняло итераций: 12
============================================================
```

Рисунок 2.9 – Результат вычисления по методу золотого сечения с помощью программы на языке Python

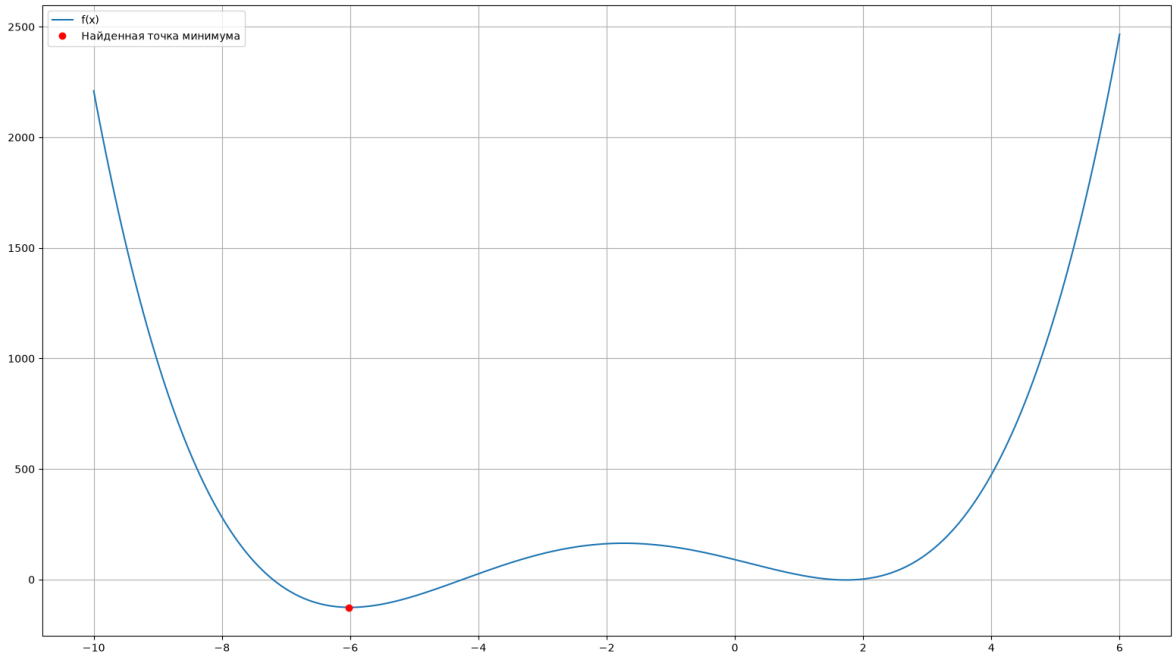


Рисунок 2.10 – График функции с найденной точкой минимума для метода золотого сечения

Задание 5

После вычисления точки минимума разными методами с помощью разных инструментов (инструментами MS Excel и с помощью языка Python) можно сделать сравнение результатов вычислений.

Сравнивая все методы можно сделать вывод, что они действительно приводят результат к теоретическому с заданной погрешностью $\varepsilon = 0{,}05$. Вычисления, произведённые с помощью инструментов MS Excel совпадают с решением задачи с помощью программы написанной на языке программирования Python, что видно как при аналитическом сравнении итоговых значений вычислений, так и визуальном сравнении на графиках.

ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения работы были приобретены практические навыки вычисления точки минимума методами одномерной минимизации унимодальной функции (с помощью методов деления отрезка пополам, Фибоначчи и золотого сечения).

Также приобретены навыки вычисления минимума функции с помощью инструментов MS Excel и посредством написания программы на языке Python.

Все полученные значения минимума функции совпадают с теоретическим расчётом с заданной погрешностью $\varepsilon = 0{,}05$.

ПРИЛОЖЕНИЕ А

Текст программы на языке Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)
pd.set_option("display.expand_frame_repr", False)

def f(x):
    return x**4 + 8 * x**3 - 6 * x**2 - 72 * x + 90

def viewPlot(flag=0, x=0, y=0):
    arr = np.linspace(-10, 6, 500)
    plt.plot(arr, f(arr), label="f(x)")
    plt.grid(True)
    if flag == 1:
        plt.plot(x, y, 'o', color="red", label="Найденная точка
минимума")
    plt.legend()
    plt.show()

COLS_BASE = ["Итерация", "a", "b", "x_1", "f(x_1)", "x_2",
"f(x_2)"]
COLS_FULL = COLS_BASE + ["x_m", "f(x_m)"]

def build_table(rows, columns):
    """Превращает список строк итераций в DataFrame и печатает
его."""
    df = pd.DataFrame(rows, columns=columns)

    formatters = {}
    for col in columns:
        if col == "Итерация":
            continue
        if col.startswith("f("):
            formatters[col] = lambda v: f"{v:.3f}"
        else:
            formatters[col] = lambda v: f"{v:.4f}"

    print(df.to_string(index=False, formatters=formatters))
    return df

def print_result(name, x_min, f_min, a, b, iterations):
    print(f"\n\nРезультат вычисления по методу {name}:")
    print(f"    Точка минимума: x_min = {x_min:.6f}")
    print(f"    Значение в точке минимума: f(x_min) =
{f_min:.6f}")
    print(f"    Интервал (границы): [{a:.6f}; {b:.6f}]")
    print(f"    Длина интервала: {np.abs(b - a):.6f}")
```

```
    print(f"    Заняло итераций: {iterations}")
    print("=========================================================
===\n\n")

def method_division(eps):
    a, b = -10, 6
    x_m = (a + b) / 2
    l = b - a
    iterations = 0
    rows = []

    while np.abs(l) >= eps:
        iterations += 1
        x_1 = a + l / 4
        x_2 = b - l / 4
        fx_1 = f(x_1)
        fx_2 = f(x_2)
        fx_m = f(x_m)

        if fx_1 < fx_m:
            b = x_m
            x_m = x_1
            l = b - a
        elif fx_2 < fx_m:
            a = x_m
            x_m = x_2
            l = b - a
        else:
            a = x_1
            b = x_2
            l = b - a

        fx_m = f(x_m)
        rows.append([iterations, a, b, x_1, fx_1, x_2, fx_2, x_m,
fx_m])

    df = build_table(rows, COLS_FULL)
    print_result("деления интервала", x_m, f(x_m), a, b,
iterations)
    viewPlot(1, x_m, f(x_m))
    return df

def method_fibonacci(eps):
    a, b = -10, 6
    l = b - a
    iterations = 0
    ratio = np.ceil(l / eps)
    x_min, f_min = 10**6, 10**6

    # Генерация чисел Фибоначчи
    F = [1, 1]
    while F[-1] < ratio:
        F.append(F[-1] + F[-2])
```

```
    n = len(F) + 1
    while len(F) <= n + 2:
        F.append(F[-1] + F[-2])

    rows = []
    while n > 1:
        iterations += 1
        x_1 = a + (F[n] / F[n + 2]) * (b - a)
        x_2 = a + (F[n + 1] / F[n + 2]) * (b - a)
        fx_1 = f(x_1)
        fx_2 = f(x_2)

        rows.append([iterations, a, b, x_1, fx_1, x_2, fx_2])

        if n == 2:
            if fx_1 <= fx_2:
                x_min, f_min = x_1, fx_1
            else:
                x_min, f_min = x_2, fx_2
            break
        else:
            if fx_1 <= fx_2:
                b = x_2
                x_2, fx_2 = x_1, fx_1
                n -= 1
                x_1 = a + (F[n] / F[n + 2]) * (b - a)
                fx_1 = f(x_1)
            else:
                a = x_1
                x_1, fx_1 = x_2, fx_2
                n -= 1
                x_2 = a + (F[n + 1] / F[n + 2]) * (b - a)
                fx_2 = f(x_2)

    df = build_table(rows, COLS_BASE)
    print_result("Фибоначчи", x_min, f_min, a, b, iterations)
    viewPlot(1, x_min, f_min)
    return df

def method_golden(eps):
    tau = 0.38197
    a, b = -10, 6
    iterations = 0

    x_1 = a + tau * (b - a)
    x_2 = b - tau * (b - a)
    fx_1, fx_2 = f(x_1), f(x_2)

    rows = []
    while np.abs(b - a) > eps:
        iterations += 1
        if fx_1 <= fx_2:
            b = x_2
```

```
            x_2, fx_2 = x_1, fx_1
            x_1 = a + tau * (b - a)
            fx_1 = f(x_1)
        else:
            a = x_1
            x_1, fx_1 = x_2, fx_2
            x_2 = b - tau * (b - a)
            fx_2 = f(x_2)

        x_m = (a + b) / 2
        rows.append([iterations, a, b, x_1, fx_1, x_2, fx_2])

    df = build_table(rows, COLS_BASE)
    x_min = (a + b) / 2
    f_min = f(x_min)
    print_result("золотого сечения", x_min, f_min, a, b,
iterations)
    viewPlot(1, x_min, f_min)
    return df

def mainloop():
    eps = 0.05
    while True:
        print("======== МЕНЮ ========")
        print("1. Метод деления интервалов")
        print("2. Метод Фибоначчи")
        print("3. Метод золотого сечения")
        print("4. Показать график функции на отрезке [-10; 6]")
        print("5. Выход")
        c = int(input("-> "))

        if c == 5:
            return
        elif c == 4:
            viewPlot(0)
        elif c == 1:
            df = method_division(eps)
        elif c == 2:
            df = method_fibonacci(eps)
        elif c == 3:
            df = method_golden(eps)
        else:
            print("\n\nНеизвестная команда\n\n")

if __name__ == "__main__":
    mainloop()
```